Institut Teknologi Bandung

Directorate of Continuing Professional Education

in partnership with

MODULE

Agentic AI

What you build around a language model — and the more useful question of when you should not.

Duration 3 hours (2 sessions)

Instructor Hendri Karisma, M.T.

Source Module material for Designing and Building AI Products and Services

LEARNING OUTCOMES

Session Objectives

By the end of this session, participants can:

1. Distinguish **AI, AI workflow, AI agent, agentic system, and multi-agent system** using one operational test.
2. Describe the **agent loop** — plan, act, observe, check — and name what terminates it.
3. Design a **tool** properly: schema, description, validation, and the fact that only your code touches a real system.
4. Choose between the **four kinds of memory**, and say which problem each one solves.
5. Lay out a **tech stack** across model, orchestration, tools, memory, guardrails, and observability.
6. Apply a **three-question test** before splitting a system into multiple agents.
7. Build an **evaluation harness** for a non-deterministic system, and explain why traces come first.
8. Place a proposed deployment on the **autonomy ladder** and justify the level chosen.

SITE
Course home

REPO
ai-agentic-demo — single-agent and multi-agent cases

01

What the words mean

Five terms that are used interchangeably and should not be.

SETTING UP

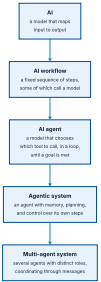
This module sits on top of chapter 16

This module asks a different question: **given such a model, what do you build around it** so that it can affect the world rather than only describe it?

> Nothing in this module makes the model more capable. Everything in it is about **what the model is connected to**, and about **what happens when it is wrong**

VOCABULARY

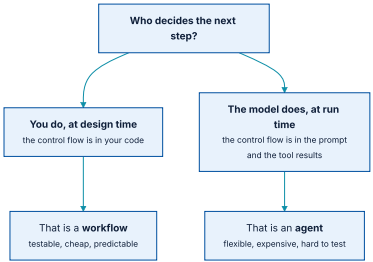
Five terms, in increasing order of autonomy



Each step gives the model more say over what happens next.

These are used interchangeably in vendor material and they are not interchangeable. **The difference between them is a cost difference, a testability difference, and a risk difference** — which makes it worth being precise.

One question separates a workflow from an agent



Everything else — the model, the tools, the prompt — can be identical.

If you can draw the flowchart in advance and it does not change per request, you have a workflow. Build it as a workflow. It will be cheaper, faster, and you will be able to test it.

VOCABULARY

The five, stated precisely

TERM	CONTROL FLOW	WHAT IT IS GOOD FOR
AI	None — one call in, one answer out	Classification, extraction, summarization, generation.
AI workflow	**Yours**, fixed at design time	Most business processes. Predictable cost and latency.
AI agent	**The model's**, chosen per step from a tool set	Tasks whose shape is not known until you see the request.
Agentic system	The model's, plus memory and planning across turns	Long-running work; tasks that resume.
Multi-agent	A supervisor routing between specialists	Genuinely separable subtasks with different tools or permissions.

A NOTE ON THE HYPE

Deflating this on purpose

Most problems brought to "agents" are workflows. Most multi-agent designs do not survive contact with a cost model.

That is not a reason to avoid the topic. It is the reason to learn it properly — because the value in this area is almost entirely in **tools, memory, and evaluation**, and almost none of it is in prompts.

A useful habit for the rest of this module: every time you see a technique, ask **what it would cost to run 10,000 times a day, and how you would know it was working**.

02

How an agent actually runs

A loop, a tool schema, and something that makes it stop.

Plan, act, observe, check

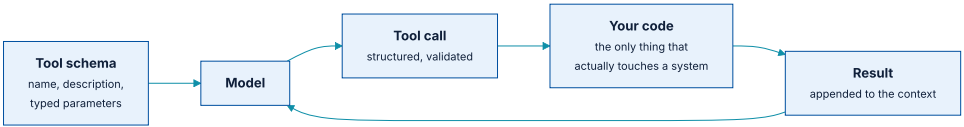
The whole of agent engineering is in what fills each of these boxes.

Note the shape. This is **the generation loop from chapter 16 with a tool call in the middle** — the model's output at one step becomes part of its input at the next. Everything chapter 15 said about that pattern still applies.

What terminates it

- **Goal satisfied** — the model says it is done, and something else checks that claim.
- **Step budget** — a hard maximum number of iterations.
- **Token or currency budget** — a hard maximum spend per request.
- **Wall-clock timeout** — because a hung tool is not the model's problem to solve.
- **Repetition detector** — the same tool with the same arguments twice in a row is almost always a loop, not progress.
- **Escalation path** — what happens when the budget is spent and the goal is not met. *Silently returning a partial answer is the worst option.*

The model never touches anything



The model emits a structured request. Your code decides whether to honour it.

This is the single most important architectural fact in the module. **The model produces text describing an intention. Your code is the only thing that executes.** Every permission boundary you have lives in that gap.

A tool is an API designed for a reader who forgets

1. **Name it for the task, not the system.** `find_customer_by_email` beats `crm_query_v2`.
2. **Write the description for the model, not for your team.** It is the only documentation the model gets, and it is read fresh every single call.
3. **Type every parameter, and validate on arrival.** Assume the arguments are adversarial, because sometimes they will be.
4. **Return errors as data, not exceptions.** A tool that returns `{"error": "no customer with that email"}` lets the agent recover; one that throws ends the run.
5. **Keep results small.** Every byte a tool returns occupies context that the model then pays to read on every subsequent step.

Most *prompt engineering* problems in agent systems are actually **tool description problems** wearing a disguise.

Read tools and write tools are not the same risk

Read tools

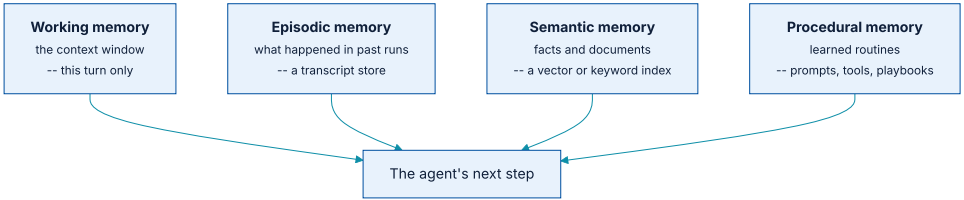
Search, retrieve, look up, calculate. A wrong call wastes tokens and returns something unhelpful. **Recoverable within the loop.**

Write tools

Send, delete, transfer, approve, publish. A wrong call **changes the world**, and no amount of subsequent reasoning undoes it.

Treat these differently from day one. **Read tools can be autonomous; write tools should start behind an approval gate** and only move out of it with evidence from your evaluation harness.

Four kinds, solving four different problems



Borrowed vocabulary from cognitive psychology, used loosely — but the four categories map onto four different pieces of infrastructure.

The mistake to avoid is treating *memory* as one thing and reaching for a vector database by reflex. **Most agents need working memory managed well and nothing else.**

What each one costs you

KIND	IMPLEMENTATION	THE FAILURE MODE IT INTRODUCES
Working	The context window itself	Overflow. Old steps silently fall out and the agent forgets its goal.
Episodic	A transcript store, keyed by session	Replaying stale state as if it were current.
Semantic	Vector or keyword index over documents — chapter 16's RAG	Retrieving confidently irrelevant context, which is worse than none.
Procedural	Versioned prompts, tool sets, playbooks	Drift between what is deployed and what was evaluated.

Each row is a system to operate. **Add them one at a time, and only when a specific observed failure demands it.**

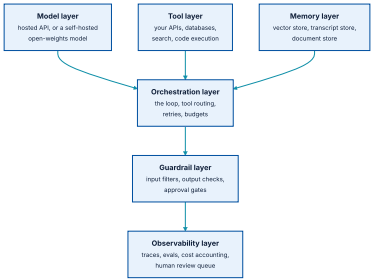
03

The tech stack

Six layers, and the two that teams skip.

ARCHITECTURE

Six layers



Guardrails and observability are the two layers most first attempts leave out entirely.

Hosted or self-hosted, and what actually decides it

Hosted API

Best capability per unit of effort, no infrastructure, immediate access to new models. **Cost scales linearly with usage**, and your data leaves your perimeter.

Self-hosted open weights

Data stays inside. Fixed infrastructure cost rather than per-token. **Lower capability at the same price point**, and you now operate a GPU fleet.

In practice the decision is rarely about capability or cost. It is about **where the data is allowed to be** — which is a regulatory question, not an engineering one, and it should be answered before any code is written.

What the orchestrator is responsible for

- **Running the loop** — and enforcing every one of the termination conditions from earlier.
- **Routing tool calls** to implementations, and validating arguments before they arrive.
- **Retrying** transient failures, with backoff, without re-charging the model for the same reasoning.
- **Accounting** — tokens, currency, and latency, per request and per tool.
- **Emitting traces** for everything above.

Frameworks exist for all of this and are worth using. But note what the list contains: **queueing, retries, budgets, and telemetry.** This is ordinary distributed-systems work, and your existing platform team already knows how to do it.

Three places to put a check

1. **On the input.** Reject or sanitise before the model sees it — prompt injection carried in a retrieved document is the case people miss.
2. **On the tool call.** Validate arguments, check permissions against the *end user's* identity rather than the service's, and gate write tools behind approval.
3. **On the output.** Schema-check structured output; scan free text for the categories your policy prohibits; verify claims against retrieved sources where you can.

> A guardrail that is itself a model call is a guardrail that can be wrong. **Prefer deterministic checks wherever the check can be expressed deterministically** — schema validation, allow-lists, permission lookups.

Observability is not optional here

A trace of every step, every tool call, and every token is the only debugging tool you have. Build it first, not after the first incident.

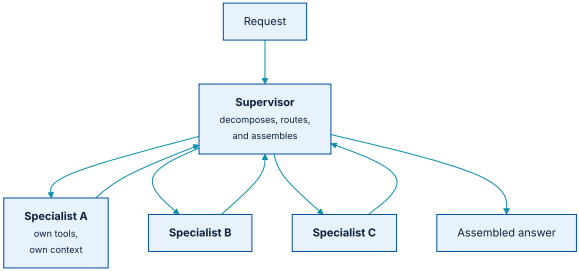
The minimum useful trace records: the full prompt sent, the model and version, every tool call with arguments and results, token counts and cost per step, latency per step, and the termination reason. **If you cannot answer *why did it do that* from your logs, you do not have logs.**

04

Multi-agent, and when not to

A supervisor, some specialists, and a three-question test.

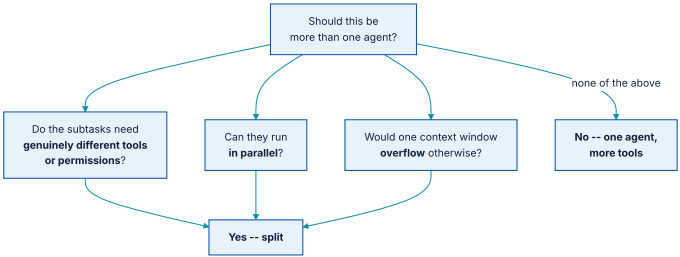
The supervisor pattern



Each specialist has its own tools, its own context, and its own instructions.

This is the pattern that appears in almost every multi-agent framework, under various names. The variations — peer-to-peer messaging, hierarchies, debate — are **elaborations of it, and each elaboration multiplies the cost.**

Three questions before you split



If none of the three is a yes, you want one agent with more tools.

Splitting for conceptual tidiness is the common mistake. Roles named *Researcher*, *Writer*, and *Critic* feel organised and typically cost three times as much as one agent doing the same work with the same tools.

What each additional agent costs

COST	WHY IT GROWS
Tokens	Every hand-off re-states context. A supervisor that summarises for three specialists pays for that summary three times, plus its own reasoning.
Latency	Sequential hand-offs add up. Parallel ones only help if the subtasks are genuinely independent.
Failure surface	Each agent can loop, drift, or hallucinate independently — **and one specialist's wrong answer becomes another's trusted input.**
Evaluation	You now need per-agent evals *and* end-to-end evals, because a system that is right overall can be right for the wrong reasons.

A finding worth carrying: decompose a task into five or six specialists and it is common for **only one or two to justify their existence** once the traces are read.

Three shapes where splitting is right

Different permissions

One agent may read customer records; another may write to the ledger. **Splitting is how you keep those capabilities apart** — a security boundary, not a stylistic one.

Genuine parallelism

Twenty documents to analyse independently. Fan out, collect, assemble. The latency win is real and the contexts do not interact.

Context that will not fit

A task whose inputs genuinely exceed the window. Splitting is a **memory strategy** here, and the supervisor's summaries are the compression.

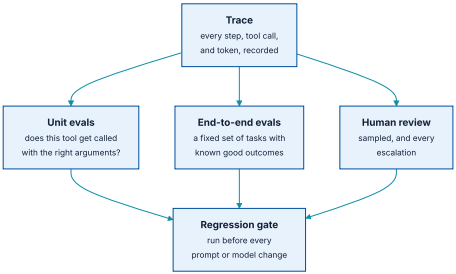
05

Evaluation and operation

How you know it works, and what you do when it stops.

EVALUATION

You cannot test this the way you test code



Traces first. Everything downstream is built on them.

What each level catches

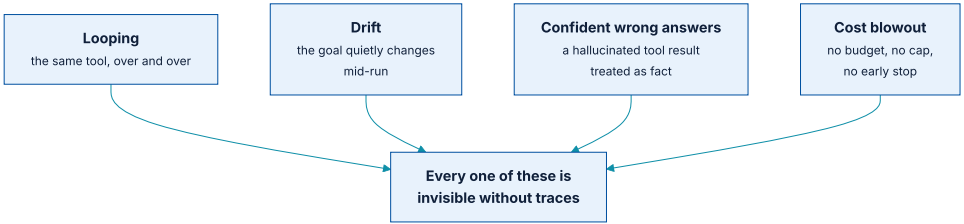
LEVEL	WHAT IT ASSERTS	WHAT IT MISSES
Unit evals	Given this input, the right tool is called with the right arguments.	Whether the sequence of correct calls produces a useful outcome.
End-to-end evals	Given this task, the final outcome matches a known good one.	*Why* it succeeded — a right answer for the wrong reason still passes.
Human review	Whether the output is actually good, by a standard you cannot encode.	Coverage. It is sampled, so it finds classes of failure, not instances.

Run all three **before every prompt change, every model upgrade, and every tool change**. In this domain a prompt edit is a code change with no type checker behind it.

Where the evaluation tasks come from

1. **Start with twenty real requests**, taken from whatever the process looks like today. Not twenty imagined ones.
2. **Write down the good outcome for each** — before you build anything, while you are still honest about what good means.
3. **Add every failure you observe** in development, permanently. This is how the set grows, and it is why it is worth versioning.
4. **Include the boring cases.** A set made only of hard cases will let you ship something that fails the easy ones.
5. **Keep a held-out slice** you do not look at while iterating. Chapter 18's warning about validation-set overfitting applies here exactly.

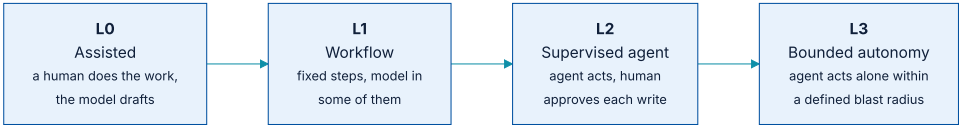
Four failure modes, and how each announces itself



None of these throw an exception. All of them show in a trace.

Set alerts on the **shape** of runs, not only on errors: mean steps per request, cost per request, tool-call repetition rate, and escalation rate. **A rising step count is usually the first visible sign that something upstream changed.**

The autonomy ladder



Move up a rung only with evidence from the evaluation harness.

At **L3**, the question that matters is not *how good is it?* but **what is the blast radius when it is wrong?** Define that boundary in code — spend limits, record scopes, reversibility — not in the prompt.

06

Cases

What we will build, and what each one is chosen to teach.

CASES

Single-agent cases

CASE	WHAT IT TEACHES
Document Q&A with citations	Retrieval as a tool, not as a preprocessing step. Grounding, and how to make a citation verifiable rather than decorative.
Structured extraction from messy input	Schema-constrained output, validation on arrival, and what to do with the 20% that fails validation.
Internal API assistant	Tool design under real permissions. Read tools autonomous, write tools gated. Identity propagation.
Data analysis with code execution	The most powerful and most dangerous tool there is. Sandboxing, resource limits, and why the output must still be checked.

Multi-agent cases

CASE	WHAT IT TEACHES
Parallel document triage	The clean case for fan-out: independent subtasks, real latency win, no shared context. The one that unambiguously justifies splitting.
Research and draft with a separate reviewer	The *tidy* decomposition — and a cost comparison against one agent doing both, run honestly.
Cross-permission workflow	Two agents that must not share capabilities. Splitting as a security boundary.
Escalation to a human	The hand-off nobody designs and everybody needs. What state goes with it, and how the human hands it back.

All four ship in the **ai-agentic-demo** monorepo, each with a trace viewer and an evaluation set, so the cost and quality claims on these slides can be checked rather than believed.

What you will do with them

1. **Bring a process from your own work** — one that exists, with real inputs you can describe.
2. **Classify it** using the operational test from section 01. Most will be workflows, and that is the correct answer.
3. **Write the twenty evaluation cases** before writing any prompt.
4. **Build the smallest version** that could work: one agent, the fewest tools that cover the task.
5. **Run it, read the traces, and report what surprised you.** That last step is the assessment.

> The deliverable is not a working agent. It is **a defensible answer to whether you should have built one** — supported by traces, costs, and an evaluation set.

SUMMARY

What has to survive this module

1. **Who decides the next step** is the question that separates a workflow from an agent. Most things are workflows; build them as workflows.
2. **The loop needs more than one exit.** Goal, steps, tokens, time, repetition, and an escalation path.
3. **The model never touches anything.** It emits a structured intention; your code decides whether to honour it. Every permission boundary lives there.
4. **Read tools and write tools are different risks**, and should have different defaults from day one.
5. **Memory is four things, not one.** Add each only when an observed failure demands it.

SUMMARY

What has to survive this module (2 of 2)

1. **Guardrails and observability are layers**, not features. Traces come before everything else, because nothing here reproduces.
2. **Three questions before splitting into multiple agents** — different permissions, real parallelism, or context that will not fit. Tidiness is not one of them.
3. **Evaluate at three levels** — unit, end-to-end, and sampled human review — and run all three before any prompt or model change.
4. **Move up the autonomy ladder on evidence**, and define the blast radius in code rather than in the prompt.

REPO

ai-agentic-demo

BACK

Chapter 16 — Text generation